

Relationships Between Entities in Entity Framework

In this article, I am going to discuss the **Relationships Between Entities in Entity Framework Database First Approach**, i.e., at the end of this article, you will understand how the entity framework manages the relationships between entities. Please read our previous article, where we discussed [Entity Framework DbSet Methods with Examples](#). In this article, we will work with the same example we created in our [Introduction to Entity Framework Database First](#) article. Please read our introduction to [Entity Framework Database First](#) article before proceeding to this article.

What are Relationships?

A common requirement in relational databases is to establish relationships between database tables or entities. And whenever we try to establish a relationship between two entities, then one of the entities will act as a Principal Entity, and the other Entity will act as the Dependent Entity. Let us first understand what is Principal Entity and Dependent Entity from the relational database point of view.

1. **Principal Entity:** The Entity containing the Primary or Unique Key properties is called Principal Entity.
2. **Dependent Entity:** The Entity which contains the Foreign key properties is called Dependent Entity.

From the Relational Database Point of view, we already know that the Relationships Between two entities or tables are established using Foreign Keys. A Foreign Key is a Key that refers to the Unique Key or Primary Key of another table. Once we establish the relationships between database tables or entities, then we can get meaningful results from the database.

According to [Database Normalization](#), we should not store all the information in a single table. We need to split the data into multiple tables, but there should be some link or relationship between the tables.

Before understanding the different types of relationships, let us first understand a few important terms which are used in database and model classes.

1. **Primary Key:** The Primary key is a column (columns in the case of composite primary key) in the database Table that Uniquely identifies each row in the table.
2. **Foreign Key:** The Foreign key is a column in a table that makes a relationship with another table. The Foreign key Column should point to the Principal Entity's Primary Key or Unique Key column.
3. **Navigation Properties:** In .NET, the Navigation properties define the type of relationships between the Entity. Based on the requirements, these properties are defined either in the Principal Entity or in Dependent Entity.

The Navigation Properties are again classified into two types. They are as follows:

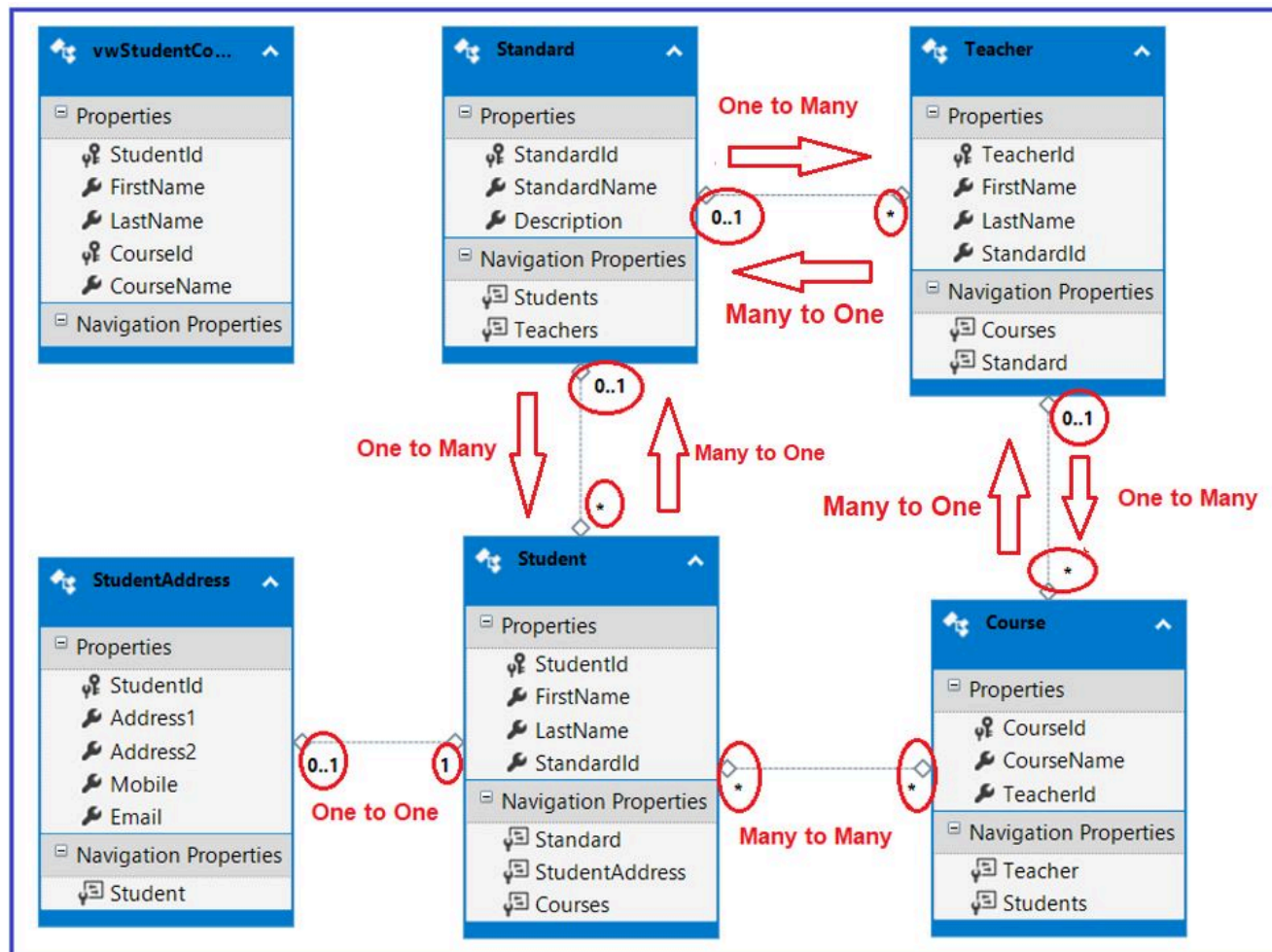
1. **Reference Navigation Property:** This is a Property that refers to a Single Related Entity, i.e., it is used to implement a One-to-One relationship between two entities.
2. **Collection Navigation Property:** This is a Property that refers to a Collection of Entities, i.e., it is used to implement One-to-Many Or Many to Many relationships between two entities.

Types of Relationships in Entity Framework:

In relational databases, there are three types of relationships between the database tables (One-to-One, One-to-Many, and Many-to-Many). The Entity Framework supports three types of relationships similar to the database. They are as follows:

1. **One-to-One Relationship**
2. **One-to-Many Relationship**
3. **Many-to-Many Relationship**

We have created an Entity Data Model for the EF_Demo_DB database in our Introduction to Entity Framework Database First Approach article. The diagram below shows the visual designer for that Entity Data Model with all the entities and the relationships between them.

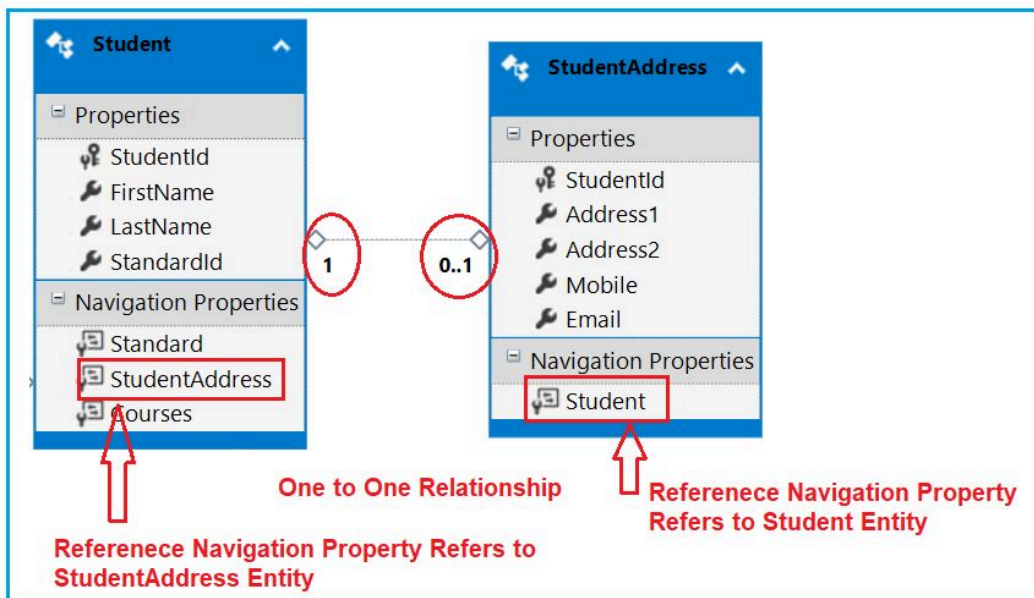


By looking at the above diagram, you might have some doubts. Let's proceed further and understand how each relationship (association) is being managed by the entity framework.

One-To-One Relationship in Entity Framework

In the One-To-One relationship, a row in Table A can have no more than one matching row in Table B, and vice versa. A one-to-one relationship is created if both of the related columns are primary keys or have unique constraints. In the one-to-one relationship, the primary key acts additionally as a foreign key, and there is no separate foreign key column.

In our EDMX File, we marked the Student and StudentAddress relationship as a One-to-One relationship (zero or one). That means a student can have only 1 or 0 addresses. Entity framework adds the Student reference navigation property into the StudentAddress entity. On the other hand, it also adds the StudentAddress navigation entity to the Student entity. For a better understanding, please have a look at the below image.



In the above image, we need to understand two things. Here, you can see in the Student Entity, it is showing 1, and in the StudentAddress Entity, it is showing 0..1. **What it means?** It means the Student Entity can have 0 or 1 Student Address. On the other hand, StudentAddress Entity belongs to a single Student Entity.

The following is the auto-generated Student Entity by the Entity Framework. You can see in the below image; it has a Reference Navigation Property called StudentAddress. This Reference Navigation Property StudentAddress establishes the one-to-one relationship between Student and StudentAddress Entities.

```
namespace DBFirstApproach
{
    using System;
    using System.Collections.Generic;

    public partial class Student
    {
        [System.Diagnostics.CodeAnalysis.SuppressMessage("Microsoft.Usage",
        "CA2214:DoNotCallOverridableMethodsInConstructors")]
        public Student()
        {
            this.Courses = new HashSet<Course>();
        }

        public int StudentId { get; set; }
        public string FirstName { get; set; }
        public string LastName { get; set; }
        public Nullable<int> StandardId { get; set; }

        public virtual Standard Standard { get; set; }

        public virtual StudentAddress StudentAddress { get; set; }

        [System.Diagnostics.CodeAnalysis.SuppressMessage("Microsoft.Usage",
        "CA2227:CollectionPropertiesShouldBeReadOnly")]
        public virtual ICollection<Course> Courses { get; set; }
    }
}
```

Student Entity Contains StudentAddress navigation Property

The following is the auto-generated StudentAddress Entity by the Entity Framework. You can see in the below image it has a Reference Navigation Property called Student. This Reference Navigation Property Student establishes the one-to-one relationship between StudentAddress and Student Entities.

```
namespace DBFirstApproach
{
    using System;
    using System.Collections.Generic;

    public partial class StudentAddress
    {
        public int StudentId { get; set; }
        public string Address1 { get; set; }
        public string Address2 { get; set; }
        public string Mobile { get; set; }
        public string Email { get; set; }

        public virtual Student Student { get; set; }
    }
}
```

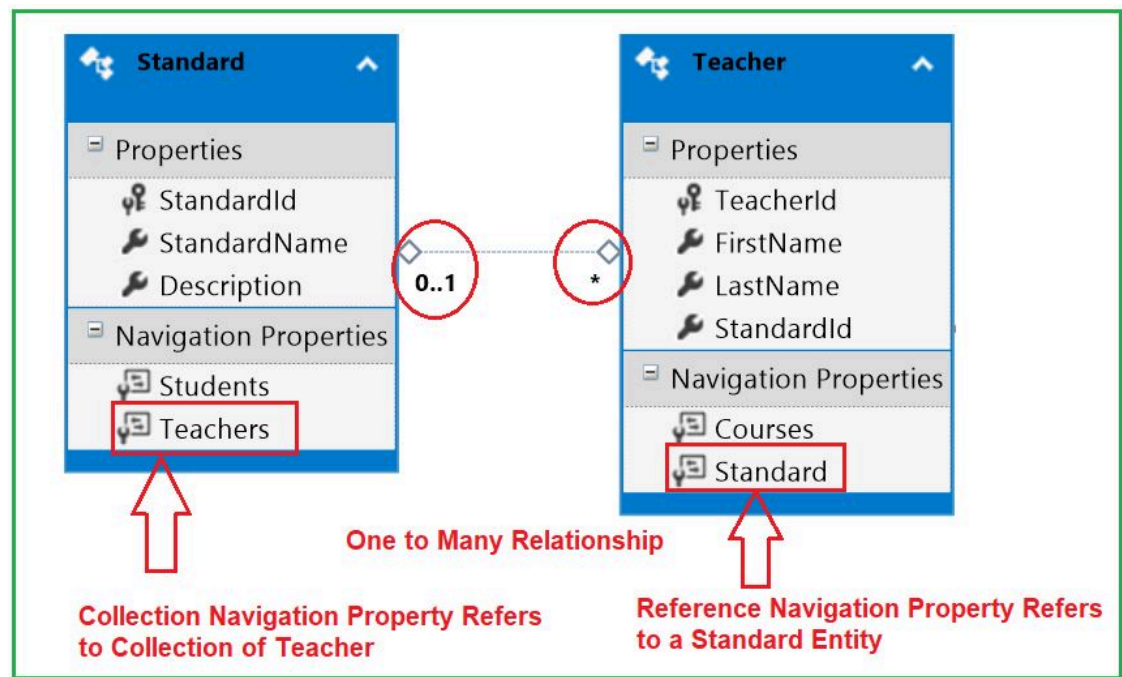
↑
StudentAddress Entity Contains Student navigation Property

Note: In the StudentAddress database table, we marked the StudentId column as both **PrimaryKey and ForeignKey**, which makes it a one-to-one relationship between the Student and StudentAddress entities.

One-to-Many Relationship in Entity Framework

The One-To-Many relationship is the most common type of relationship between database tables or entities. In this type of relationship, a row in one table, let's say TableA, can have many matching rows in another table, let's say TableB, but a row in TableB can have only one matching row in TableA.

In our example, we have marked the relationship between Standard and Teacher entities as a One-to-Many relationship. That means one Standard can have many Teachers. Here, the Entity framework adds the Collection Navigation Property Teacher inside the Standard Entity. On the other hand, it adds the Standard Reference Navigation Property inside the Teachers entity. That means one Standard can have many Teachers, whereas one Teacher can associate with only 1 or 0 Standards. For a better understanding, please have a look at the below image.



In the above diagram, we need to understand two things. Here, you can see in the Standard Entity, it is showing 0..1, and in the Teacher Entity, it is showing *. **What does it mean?** It means the Teacher Entity can belong to 0 or 1 Standard. On the other hand, Standard Entity can have many teachers.

To represent the One-To-Many relationships, the Standard entity has the Collection Navigation Property Teachers (please notice that it is in plural form), which indicates that one Standard can have a collection of Teachers (many teachers). For a better understanding, please have a look at the following auto-generated Standard entity model.

```
namespace DBFirstApproach
{
    using System;
    using System.Collections.Generic;

    public partial class Standard
    {
        [System.Diagnostics.CodeAnalysis.SuppressMessage("Microsoft.Usage",
        "CA2214:DoNotCallOverridableMethodsInConstructors")]
        public Standard()
        {
            this.Students = new HashSet<Student>();
            this.Teachers = new HashSet<Teacher>();
        }

        public int StandardId { get; set; }
        public string StandardName { get; set; }
        public string Description { get; set; }

        [System.Diagnostics.CodeAnalysis.SuppressMessage("Microsoft.Usage",
        "CA2227:CollectionPropertiesShouldBeReadOnly")]
        public virtual ICollection<Student> Students { get; set; }
        [System.Diagnostics.CodeAnalysis.SuppressMessage("Microsoft.Usage",
        "CA2227:CollectionPropertiesShouldBeReadOnly")]
        public virtual ICollection<Teacher> Teachers { get; set; }
    }
}
```


Standard entity has the collection navigation property Teachers

On the other hand, the Teacher entity has a Standard Reference Navigation Property, which indicates that the Teacher is associated with 1 or 0 Standard. The following is the auto-generated Teacher Entity Model by Entity Framework.

```

namespace DBFirstApproach
{
    using System;
    using System.Collections.Generic;

    public partial class Teacher
    {
        [System.Diagnostics.CodeAnalysis.SuppressMessage("Microsoft.Usage",
        "CA2214:DoNotCallOverridableMethodsInConstructors")]
        public Teacher()
        {
            this.Courses = new HashSet<Course>();
        }

        public int TeacherId { get; set; }
        public string FirstName { get; set; }
        public string LastName { get; set; }
        public Nullable<int> StandardId { get; set; }

        [System.Diagnostics.CodeAnalysis.SuppressMessage("Microsoft.Usage",
        "CA2227:CollectionPropertiesShouldBeReadOnly")]
        public virtual ICollection<Course> Courses { get; set; }

        public virtual Standard Standard { get; set; }
    }
}

```



Teacher entity has a Standard navigation property

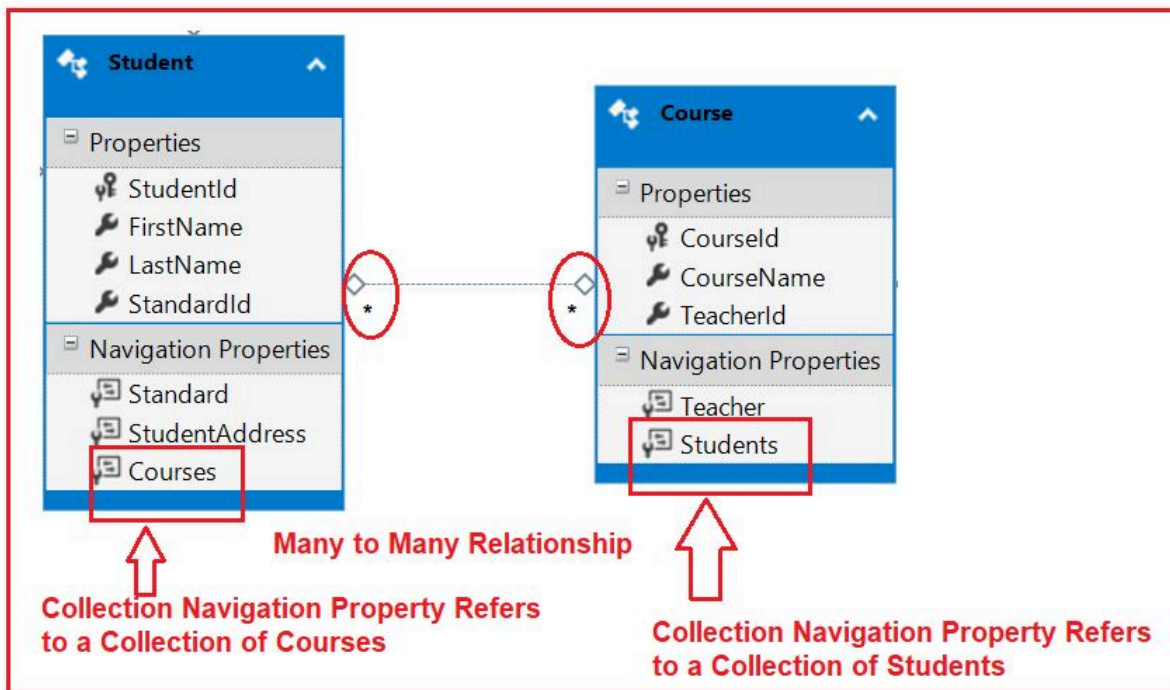
Note: In the Teacher database table, we marked the StandardId column as ForeignKey, which points to the StandardId column of the Standard table, which makes it a One-To-Many relationship between the Teacher and Standard Entities.

We also have one-to-many relationships between Standard and Student (one Standard can have many Students whereas one Student can associate with only 1 or 0 Standard), Teacher and Course (one Teacher can have many Courses whereas one Course can associate with only 1 or 0 Teacher).

Many-to-Many Relationships in Entity Framework

In many-to-many relationships, a row in one table, let's say TableA, can have many matching rows in another table, let's TableB, and vice versa is also true i.e. a row in TableB can have many matching rows in TableA. In Relational Databases, We can create many-to-many relationships by defining a third table, whose primary key consists of the foreign keys from tableA and tableB. So, here, the Primary Key is a composite Primary key.

For example, the Student and Course tables have many-to-many relationships. That means one Student can enroll in many Courses, and many students can take one Course. Here, * represents many. For a better understanding, please have a look at the below image.



In the database, we have the **StudentCourse** joining table, which includes the Primary Key of both the Student and Course tables as a composite primary key. The Entity Framework represents many-to-many relationships by not having the entity set (DbSet property) for the joining table (in our case, the joining table is StudentCourse) in the CSDL and Visual Designer. Instead, it manages this through C-S Mapping.

The Student entity includes the collection navigation property Courses and the Course entity, which includes the collection navigation property Students to represent the many-to-many relationship between them.

The following is the auto-generated Student Entity Model by the Entity Framework. You can see in the below image; it has the Courses Collection Navigation Property.

```

namespace DBFirstApproach
{
    using System;
    using System.Collections.Generic;

    public partial class Student
    {
        [System.Diagnostics.CodeAnalysis.SuppressMessage("Microsoft.Usage",
"CA2214:DoNotCallOverridableMethodsInConstructors")]
        public Student()
        {
            this.Courses = new HashSet<Course>();
        }

        public int StudentId { get; set; }
        public string FirstName { get; set; }
        public string LastName { get; set; }
        public Nullable<int> StandardId { get; set; }

        public virtual Standard Standard { get; set; }
        public virtual StudentAddress StudentAddress { get; set; }
        [System.Diagnostics.CodeAnalysis.SuppressMessage("Microsoft.Usage",
"CA2227:CollectionPropertiesShouldBeReadOnly")]
        public virtual ICollection<Course> Courses { get; set; }
    }
}

```

↑

Student Entity has Course collection navigation property

The following is the auto-generated Course Entity Model by the Entity Framework. You can see in the below image; it has the Students collection navigation property.

```

namespace DBFirstApproach
{
    using System;
    using System.Collections.Generic;

    public partial class Course
    {
        [System.Diagnostics.CodeAnalysis.SuppressMessage("Microsoft.Usage",
        "CA2214:DoNotCallOverridableMethodsInConstructors")]
        public Course()
        {
            this.Students = new HashSet<Student>();
        }

        public int CourseId { get; set; }
        public string CourseName { get; set; }
        public Nullable<int> TeacherId { get; set; }

        public virtual Teacher Teacher { get; set; }
        [System.Diagnostics.CodeAnalysis.SuppressMessage("Microsoft.Usage",
        "CA2227:CollectionPropertiesShouldBeReadOnly")]
        public virtual ICollection<Student> Students { get; set; }
    }
}

```

↑
It has Students collection navigation property

Note: The Entity Framework supports many-to-many relationships only when the joining table (StudentCourse in this case) does NOT include any columns other than PKs of both the tables, i.e., Student and Course. If the join tables contain additional columns, then the EDM creates an entity for the middle table, and you will have to manage CRUD operations for many-to-many entities manually.

How to View the StudentCourse Mapping?

Open Entity Data Model in XML view. You can see that SSDL (storage schema) has the StudentCourse entity set, as shown in the below image.

```

<edmx:Edmx Version="3.0" xmlns:edmx="http://schemas.microsoft.com/ado/2009/11/edmx"
  <!-- EF Runtime content -->
  <edmx:Runtime>
    <!-- SSDL content -->
    <edmx:StorageModels>
      <Schema Namespace="EF_Demo_DBModel.Store" Provider="System.Data.SqlClient"
        <EntityType Name="Course">...</EntityType>
        <EntityType Name="Standard">...</EntityType>
        <EntityType Name="Student">...</EntityType>
        <EntityType Name="StudentAddress">...</EntityType>
        <EntityType Name="StudentCourse">
          <Key>
            <PropertyRef Name="StudentId" />
            <PropertyRef Name="CourseId" />
          </Key>
          <Property Name="StudentId" Type="int" Nullable="false" />
          <Property Name="CourseId" Type="int" Nullable="false" />
        </EntityType>
        <EntityType Name="Teacher">...</EntityType>

```

The CSDL doesn't have the StudentCourse Entity Set. Instead, it's being mapped in the navigation property of the Student and Course entities. MSL (C-S Mapping) has a mapping between Student and Course into the StudentCourse table in the <AssociationSetMapping/> section, as shown in the image below.

```

<!-- C-S mapping content -->
<edmx:Mappings>
  <Mapping Space="C-S" xmlns="http://schemas.microsoft.com/ado/2009/11/mapping/cs">
    <EntityContainerMapping StorageEntityContainer="EF_Demo_DBModel.Store" CdmEntityContainer="EF_Demo_DBEntities">
      <EntitySetMapping Name="Courses">...</EntitySetMapping>
      <EntitySetMapping Name="Standards">...</EntitySetMapping>
      <EntitySetMapping Name="Students">...</EntitySetMapping>
      <EntitySetMapping Name="StudentAddress">...</EntitySetMapping>
      <EntitySetMapping Name="Teachers">...</EntitySetMapping>
      <EntitySetMapping Name="vwStudentCourse">...</EntitySetMapping>
      <AssociationSetMapping Name="StudentCourse" TypeName="EF_Demo_DBModel.StudentCourse" StoreEntitySet="StudentCourse">
        <EndProperty Name="Course">
          <ScalarProperty Name="CourseId" ColumnName="CourseId" />
        </EndProperty>
        <EndProperty Name="Student">
          <ScalarProperty Name="StudentId" ColumnName="StudentId" />
        </EndProperty>
      </AssociationSetMapping>
      <FunctionImportMapping FunctionImportName="spDeleteStudent" FunctionName="EF_Demo_DBModel.Store.spDeleteStudent" />
      <FunctionImportMapping FunctionImportName="spGetCoursesByS" FunctionName="EF_Demo_DBModel.Store.spGetCoursesByS" />
      <FunctionImportMapping FunctionImportName="spInsertStudent" FunctionName="EF_Demo_DBModel.Store.spInsertStudent" />
      <FunctionImportMapping FunctionImportName="spUpdateStudent" FunctionName="EF_Demo_DBModel.Store.spUpdateStudent" />
    </EntityContainerMapping>
  </Mapping>
</edmx:Mappings>

```

In the next article, I am going to discuss [How to Perform CRUD Operations using the Entity Framework](#) with Examples. In this article, I try to explain the **Relationships Between Entities in the Entity Framework**, and I hope you enjoyed this relationship Between Entities in the Entity Framework article. Please give your valuable feedback and suggestions about this article.